

CHAPTER 1. METHODOLOGY

1.1 Research Design

This paper, “FeatureSAGE: Stable Attribute Group Editing With Integrated StyleFeatureEditor Encoder for Enhanced Few-Shot Image Generation,” adopts a quantitative approach with an experimental research design to evaluate improvements to the existing SAGE framework. In particular, the study integrates the GAN inversion module of SAGE with StyleFeatureEditor (SFE), incorporating its Feature Editor module to enhance latent inversion and attribute editing functionality under a one-shot setting, where attribute manipulation is conditioned on a single reference image per test instance. The research employs a quantitative methodology using established automated metrics for objective evaluation. Reconstruction quality is assessed using Fréchet Inception Distance (FID) and Learned Perceptual Image Patch Similarity (LPIPS).

1.1.1 Experimental Setup

The experimental setup is designed to evaluate the effectiveness of the proposed FeatureSAGE framework in comparison to the existing SAGE method. The evaluation follows a quantitative, experimental research design that involves controlled changes to a single independent variable and the measurement of multiple dependent variables using standardized metrics.

- **Independent Variable**

- GAN Inversion Method: The primary manipulation involves substituting the original Pixel2Style2Pixel (pSp) encoder used in SAGE with the proposed StyleFeatureEditor (SFE) encoder, which includes an integrated feature editing module for enhanced semantic attribute control.

- **Dependent Variable** To objectively evaluate the effectiveness of the GAN inversion and

editing pipeline, the following metrics are employed:

- Reconstruction Quality

1. Fréchet Inception Distance (FID): Measures distribution-level similarity between generated and real images.
2. Learned Perceptual Image Patch Similarity (LPIPS): Measures perceptual similarity between original and reconstructed images based on deep features.

- **Controlled Variables**

1. Generator Architecture: A fixed, pretrained StyleGAN2 generator is used in both configurations for consistent image synthesis capabilities.
2. Training Dataset: Both configurations use the same datasets, FUNIT Animal Faces and 102 Flowers, with the same preprocessing and data-splitting.
3. Evaluation Protocol: The same unseen categories from the train-test split data are used for one-shot testing in both configurations. Evaluation is conducted using identical random seeds, sample counts, and query sets.
4. Loss Functions: Reconstruction and editing losses are applied consistently across both systems.
5. Optimization Settings: Both models utilize the same optimizer type, learning rate, batch size, and training epochs.
6. Evaluation Metrics: All quantitative metrics, FID and LPIPS, are computed using the same implementation and parameters for consistency.
7. Hardware and Software Environment: All experiments are conducted using the same hardware and PyTorch environment to avoid performance inconsistencies. The system specifications are reported in Section 1.2.

- **Experimental Conditions**

Two experimental configurations are used:

- SAGE (Baseline): Employs the original pSp encoder with StyleGAN2.
- FeatureSAGE (Proposed): Integrates StyleFeatureEditor (SFE) encoder and a Feature Editor.

1.1.2 Few-Shot Experimental Setting

In this study, the term *few-shot* refers to the number of reference images available per target class or identity used to construct class-level latent representations for attribute group editing and downstream evaluation. A one-shot setting utilizes a single reference image per target class, representing an extreme data-scarce scenario in which attribute editing and inversion must be performed with minimal visual information. This one-shot configuration is applied consistently across all datasets and experimental conditions for both the baseline SAGE and the proposed FeatureSAGE framework. By explicitly controlling the evaluation to a single reference image per class, the experimental protocol enables a systematic assessment of FeatureSAGE’s robustness, stability, and generalization capability under severe data limitations. The few-shot constraint is applied during test-time adaptation and evaluation on unseen classes, while the StyleGAN2 generator and attribute directions are learned from large-scale seen-category data.

1.2 Hardware and System Requirements

All experiments were conducted on a virtual machine instance with GPU pass-through hosted on the Caraga State University ICTC supercomputer server. The same hardware and software environment was used for the baseline and proposed configurations so that differences in FID and LPIPS could be attributed to the inversion and editing pipeline rather than to system-level variation.

- **CPU:** QEMU Virtual CPU version 2.5+ with two processors operating at 2.19 GHz.
- **RAM:** 32.0 GB system memory.

- **GPU:** NVIDIA GRID V100D-32A, a virtualized Tesla V100 GPU with 16 GB HBM2 memory.
- **Storage:** 1 TB server HDD storage. The environment stores the FUNIT Animal Faces and 102 Flowers datasets, pretrained StyleGAN2 generator checkpoints, pSp and StyleFeatureEditor checkpoints, generated images, and evaluation outputs.
- **Software environment:** The implementation used Linux Ubuntu 24.04 LTS with a PyTorch environment, a pretrained StyleGAN2 generator, the pSp encoder baseline, the StyleFeatureEditor Inverter and Feature Editor, MTCNN-based preprocessing for Animal Faces, Adam optimization, mixed-precision training, Inception-v3 features for FID, and deep network features for LPIPS.

The 16 GB V100-class GPU is particularly important because FeatureSAGE performs multiple generator and inversion passes and injects intermediate feature tensors during synthesis. Mixed precision was enabled during Feature Editor training to reduce memory usage while preserving the same experimental protocol across comparisons.

1.3 Data Preprocessing

1.3.1 Data Sources

This research draws from a range of publicly available datasets with a varied scope of visuals such as animal faces and flowers. The datasets are popular for generating and attribute-editing applications because of their quality and well-labeled categories. For the purpose of this research’s emphasis on few-shot image synthesis, only a minority of the images from each dataset are employed. Specifically, a minimal set of representative samples per class are sampled to replicate the few-shot learning situation. The datasets used are:

1. FUNIT Animal Faces Dataset

Figure 1-1 shows representative samples from the FUNIT Animal Faces dataset. The dataset is comprised of 117,484 512×512 resolution high-quality animal face images and is



Figure 1-1. Sample images from the FUNIT Animal Faces dataset. Source: (?)

organized in 149 classes, where each class is one animal species. Each domain includes a diverse range of species and breeds, ensuring variation in appearance while maintaining consistent facial alignment to support generative model training. The images were curated from openly available sources and standardized in resolution and alignment, with the eyes centrally positioned to provide consistency for learning. A subset of the images is reserved as a test set, with the remainder used for training. The diversity and quality of the FUNIT Animal Faces dataset make it a strong benchmark for generative and attribute-editing models in few-shot settings (?).

2. 102 Flower (Flowers)



Figure 1-2. Sample images from 102 Flower Dataset. Source: (?)

Figure 1-2 shows sample images from the 102 Flower dataset, which comprises 102 flower classes present in the United Kingdom. There are between 40 and 258 images in each class. The flowers covered by the dataset are regularly occurring species in the UK. The dataset shows a great deal of variation in the sizes, pose, and lighting conditions of the flowers, which is an added complexity for the task of classification. Moreover, there are large intra-class differences for some categories, and there are many categories that are visually extremely similar to each other. All such attributes make the dataset extremely suitable for fine-grained visual categorization. The dataset was also used for

analysis and visualization using isomap dimensionality reduction based on shape and color descriptors, which revealed the structure of the different flower categories. As for the sizes of the images, the raw images in the dataset are of varying sizes. For instance, one raw image from the dataset measures 500×667 pixels (?).

1.3.2 Data Preprocessing Steps

For all datasets, the following preprocessing steps are applied:

- **Resizing and Normalization:** Images are resized according to the generator and evaluation setting, including the reported 1024×1024 experiments where applicable, and pixel values are normalized to the range $[-1, 1]$ for input to StyleGAN2.
- **Filtering:** Extremely low-quality or mislabeled images are removed.
 - For FUNIT Animal Faces, MTCNN (Multi-Task Cascaded Convolutional Networks) is used to crop and horizontally align faces so the eyes are leveled.
- **Train-Test Splitting:** The approach is assessed on few-shot image domains by partitioning each dataset into seen classes for model fitting and unseen classes for testing.
 - **Animal Faces.** Out of the total categories, 119 are used as seen classes for training, while the remaining 30 serve as unseen classes for evaluation.
 - **Flowers.** The study assigns 85 categories to the training, or seen, split and reserves 17 categories for the test, or unseen, split.
- **Augmentation:** No data augmentation is used beyond centering and flipping, since the evaluation focuses on the given domains. All datasets and preprocessing pipelines are documented for reproducibility.

1.4 Model Architecture

This study proposes a modified SAGE framework named **FeatureSAGE**, which integrates several modules to enhance few-shot image generation. The architecture combines a pretrained StyleGAN2 backbone, a StyleFeatureEditor Inverter and Feature Editor, and SAGE’s attribute group editing mechanism. The design aims to enable fine-grained, disentangled, and class-consistent image synthesis from limited input data. The main modules are as follows:

1. **StyleGAN2 Generator:** As in SAGE, the synthesis module of a pretrained StyleGAN2 generator is used to generate images from inverted latent codes. In FeatureSAGE, the generator also receives edited intermediate feature tensors from the Feature Editor so that the output image is conditioned by both latent-space edits and feature-level details.
2. **StyleFeatureEditor Inverter and Feature Editor:** StyleFeatureEditor (SFE) by ? enables editing in both W^+ latents and F -latents. This allows the model to reconstruct finer image details and preserve them during editing. In FeatureSAGE, SFE is integrated into the GAN inversion module $I(\cdot)$ of the original SAGE framework to improve latent inversion and attribute editing. According to ?, reconstruction with pSp, ReStyle, and HyperStyle may lose local details, reducing the accuracy of reconstructed images by more than 5% compared with standard data. This motivates the use of the StyleFeatureEditor Inverter $I(\cdot)$ and Feature Editor $H(\cdot)$ as the inversion and feature-editing components of the proposed framework.

1.4.1 Feature Extraction and Latent Representation

FeatureSAGE extracts three types of representations before image synthesis. First, each input image is encoded into a W^+ latent code. The W^+ space contains layer-specific StyleGAN2 style vectors; in this study, it is represented as 18 layers with 512-dimensional style information per layer. This representation is important because it supports semantic attribute editing while preserving the layer-wise structure used by StyleGAN2.

Second, the StyleFeatureEditor Inverter predicts an intermediate feature tensor F_k , where k denotes the selected StyleGAN2 generator layer used for feature injection. In the decoder block of this study, the feature tensor at layer 9 is used. The feature tensor preserves local visual details that may be lost when editing only in W^+ , such as texture, color boundaries, and fine structural information.

Third, the framework computes class-level and attribute-level latent representations. For a category C with N_C available latent codes, the class embedding is the mean latent representation:

$$e^C = \frac{1}{N_C} \sum_{i=1}^{N_C} w_i^C. \quad (1)$$

Equation (1) represents the category-relevant component because it summarizes the shared structure of a class. The residual between an image latent and its class embedding is then used to represent category-irrelevant variation, such as pose, color, texture, orientation, and background. These residuals are the basis for attribute factorization and adaptive editing.

The training and test sets are denoted as

$$D_{\text{train}} = \{x_i^{C_s}\}_{i=1}^S, \quad D_{\text{test}} = \{x_i^{C_u}\}_{i=1}^U, \quad (2)$$

where C_s denotes seen categories, C_u denotes unseen categories, S is the number of seen-category samples used in training, and U is the number of unseen-category samples used for testing. Equation (2) defines the data split used by the one-shot evaluation protocol.

1.4.2 Framework Flow

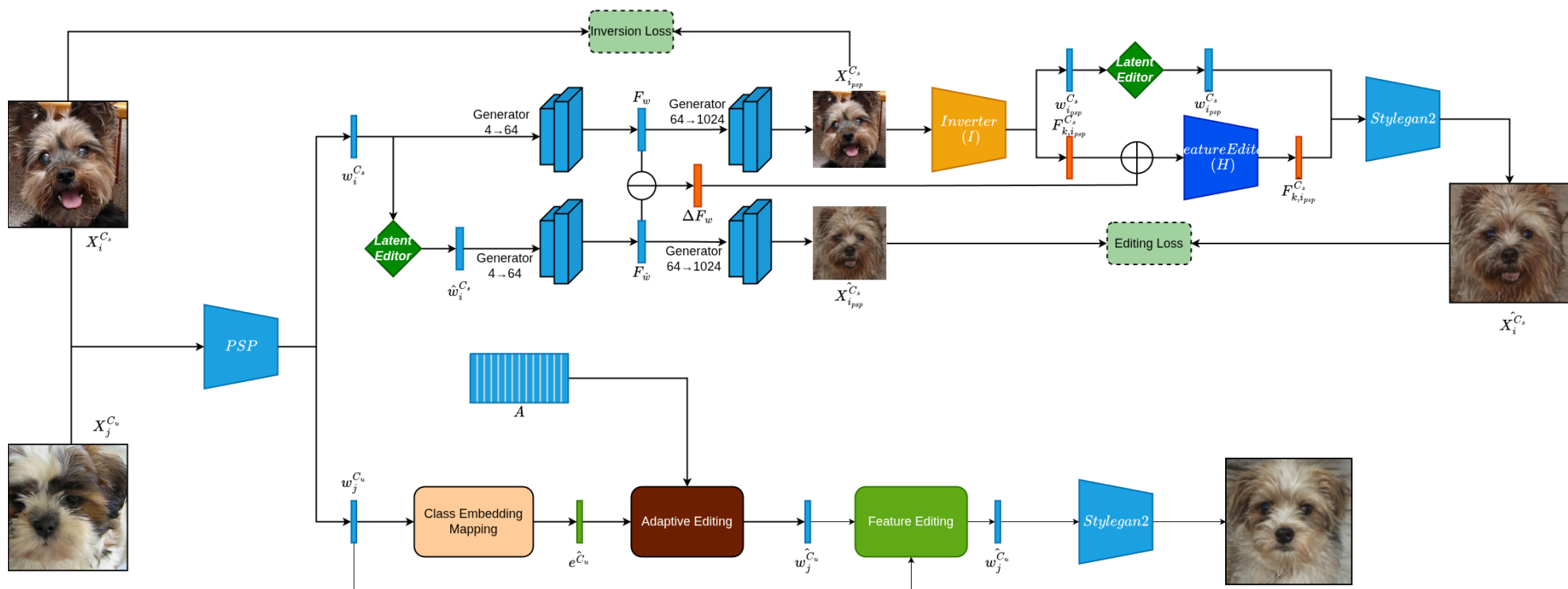


Figure 1-3. FeatureSAGE framework

Figure 1-3 shows the complete FeatureSAGE framework. The pipeline follows a sequential input-processing-output structure:

1. **Input:** A one-shot reference image from an unseen category is preprocessed through resizing, alignment when applicable, and normalization before inversion.
2. **Latent and feature extraction:** The image is encoded into W^+ latent codes and intermediate feature tensors. The pSp encoder is used in the baseline path, while the proposed path uses the StyleFeatureEditor Inverter.
3. **Attribute factorization:** Seen-category latent residuals are decomposed into category-relevant and category-irrelevant components. The category-irrelevant directions are stored in dictionary A .
4. **Class embedding mapping:** The class-level latent representation is filtered to estimate the category-relevant embedding of unseen categories.
5. **Adaptive editing:** A category-specific subset of attribute directions is selected and sampled to produce stable edits for the unseen category.
6. **Decoder and output synthesis:** The edited W^+ latent and the edited intermediate feature tensor are passed to the StyleGAN2 generator. The output is a generated image that preserves class-relevant identity while modifying category-irrelevant attributes.

For the baseline training pair, a pretrained pixel2style2pixel encoder pSp takes a seen image $X_i^{C_s}$ and predicts a W^+ latent code:

$$w_i^{C_s} = \text{pSp}(X_i^{C_s}), \quad w_i^{C_s} \in W^+. \quad (3)$$

Equation (3) defines the pSp inversion step used to obtain the baseline latent code. In FeatureSAGE, the StyleFeatureEditor Inverter predicts both the latent code and intermediate feature tensor:

$$I(X_i^{C_s}) = (w_{i,\text{sfe}}^{C_s}, F_{k,i}^{C_s}). \quad (4)$$

Equation (4) is used because SFE explicitly represents both global style information through $w_{i,\text{sfe}}^{C_s}$ and local detail through $F_{k,i}^{C_s}$.

Using the latent editor, random directions are sampled from the sparse dictionary A to obtain an edited latent code $\hat{w}_i^{C_s}$. The original and edited latent codes are then synthesized by the StyleGAN2 generator:

$$(x_{i,\text{psp}}^{C_s}, F_w) = G(w_i^{C_s}), \quad (\hat{x}_{i,\text{psp}}^{C_s}, F_{\hat{w}}) = G(\hat{w}_i^{C_s}). \quad (5)$$

Equation (5) provides the editing pair used in Feature Editor training. The images $x_{i,\text{psp}}^{C_s}$ and $\hat{x}_{i,\text{psp}}^{C_s}$ supply the reconstruction and edited targets, while F_w and $F_{\hat{w}}$ provide the feature-space difference used by the decoder.

1.4.3 Attribute Factorization

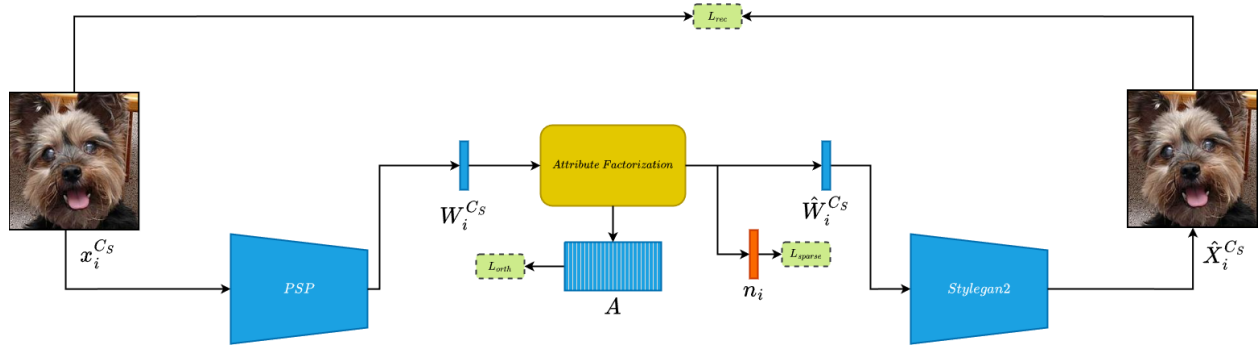


Figure 1-4. Attribute Factorization Training Framework (?)

To enable disentangled editing, FeatureSAGE retains the SAGE attribute factorization stage. This stage learns category-irrelevant attributes in the latent space by modeling the difference between each sample latent code and the mean latent code of its category.

In the Attribute Factorization block shown in Figure 1-5, the residual latent component is computed as

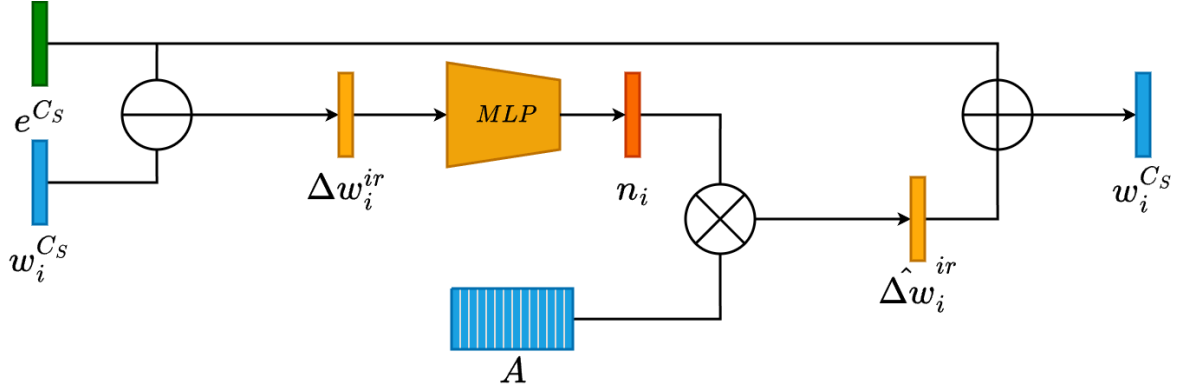


Figure 1-5. Attribute Factorization Block, derived from (?)

$$\Delta w_i^{ir} = w_i^{Cs} - e^{Cs}. \quad (6)$$

Equation (6) subtracts the category embedding from the image latent code, leaving variation that is not essential to the category identity. The residual is then approximated through dictionary directions and sparse coefficients:

$$\widehat{\Delta w}_i^{ir} = A n_i. \quad (7)$$

In Equation (7), A is the global dictionary of category-irrelevant attribute directions and n_i is the sparse code that selects which directions are active for sample i . The edited residual is added back to the class embedding to obtain an edited latent code.

Category-irrelevant attributes are features that influence how an object appears but are not essential to its identity or class membership. For example, a cat remains a cat even if its color, orientation, background, texture, or expression changes. Following ?, category-irrelevant editing is defined as

$$\text{edit}_{ir}(G(W_i^{Cs})) = G(W_i^{Cs} + \alpha \Delta W^{ir}, H(F_{k,i}, \Delta W^{ir})) = \hat{X}_i^{Cs}. \quad (8)$$

Equation (8) expresses the main editing operation. The scalar α controls edit strength, ΔW^{ir} is the category-irrelevant direction, $H(\cdot)$ edits the feature tensor, and $\hat{X}_i^{Cs}$ is the edited

output image.

Given a latent code $W_i^{C_s}$ and its class embedding e^{C_s} , the category-irrelevant direction sample is obtained by

$$\Delta W_i^{ir} = W_i^{C_s} - e^{C_s}. \quad (9)$$

Equation (9) is the same residual principle as Equation (6), written for the W^+ latent tensor.

According to ?, a global dictionary $A \in \mathbb{R}^{18 \times 512 \times l}$ and a sparse representation $n_i \in \mathbb{R}^{18 \times l}$ are learned from ΔW_i^{ir} using sparse dictionary learning:

$$\min_{n_i} \|n_i\|_0 \quad \text{s.t.} \quad \Delta W_i^{ir} = A n_i. \quad (10)$$

Equation (10) uses the L_0 constraint to encourage each sample to activate only a small number of dictionary directions, making each direction more semantically meaningful.

In practice, the sparse code is estimated through a multilayer perceptron:

$$n_i = \text{MLP}(\Delta W_i^{ir}). \quad (11)$$

Equation (11) makes the sparse dictionary learning step differentiable and trainable within the attribute factorization module.

1.4.4 Class Embedding Mapping

Compared with Attribute Group Editing (AGE) by ?, which randomly samples editing directions from A , SAGE uses class embedding mapping. The class embedding mapping block in Figure 1-6 identifies the latent directions that have the strongest effect on the target attribute group and uses them to guide later edits. This part of the framework remains unchanged from SAGE.

Let B denote the set of class embeddings from S seen categories. Since these embeddings are based on average latent codes, they may still contain minor interference from category-irrelevant attributes. Singular value decomposition (SVD) is applied to identify the most

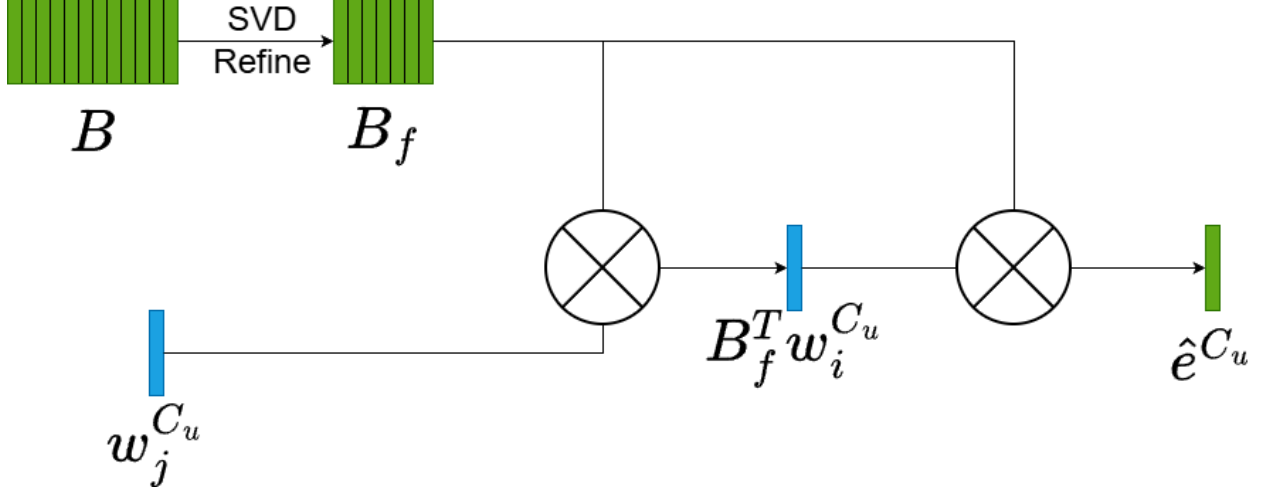


Figure 1-6. Class Embedding Block, adapted from (?)

informative directions:

$$B = U_B \Sigma_B V_B^*. \quad (12)$$

In Equation (12), U_B contains the left singular vectors, Σ_B contains the singular values in descending order, and V_B^* is the conjugate transpose of the right singular vector matrix. The top- t_B directions in U_B are selected as the final category-relevant attribute dictionary $B_f \in \mathbb{R}^{18 \times 512 \times t_B}$.

For an unseen category C_u , the class embedding is estimated by projecting its latent codes onto B_f and averaging:

$$\hat{e}^{C_u} = \frac{1}{N_u} \sum_{i=1}^{N_u} B_f B_f^T W_i^{C_u}. \quad (13)$$

Equation (13) defines $\hat{e}^{C_u}$ as the category-relevant embedding of unseen category C_u , where N_u is the number of available unseen-category latent codes and $\{W_i^{C_u}\}_{i=1}^{N_u}$ are the corresponding latent representations.

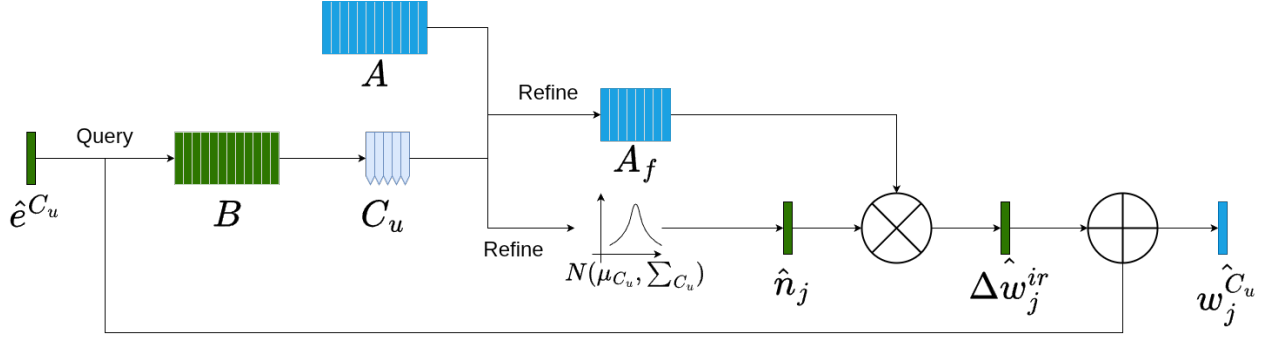


Figure 1-7. Adaptive Editing Block, from (?)

1.4.5 Adaptive Editing

Figure 1-7 shows the adaptive editing block of FeatureSAGE. Category-irrelevant qualities often have similar distributions in closely related categories but differ across distant categories. Therefore, adaptive editing selects the top- t_C seen categories whose class embeddings are closest to the unseen category embedding. To find the category-irrelevant editing directions that best fit C_u , the residual direction is back-projected into sparse-code space:

$$\hat{n}_i = A^\dagger \Delta W_i^{ir}. \quad (14)$$

In Equation (14), $A^\dagger$ denotes the pseudo-inverse of A , and $\hat{n}_i$ estimates how strongly each dictionary direction contributes to sample i .

The mean absolute activation across the selected similar seen categories is then computed as

$$\overline{|n|}^{C_u} = \frac{1}{t_C} \sum_{t=1}^{t_C} \frac{1}{N_{S_t}} \sum_{i=1}^{N_{S_t}} |\hat{n}_i^{C_{S_t}}|. \quad (15)$$

Equation (15) measures how frequently or strongly dictionary directions appear across categories similar to C_u . For each layer in W^+ , the top- t_A directions with the highest values in $\overline{|n|}^{C_u}$ are selected to form the adaptive category-irrelevant dictionary $A_{C_u} \in \mathbb{R}^{18 \times 512 \times t_A}$.

Assuming that the sparse representations of similar categories follow a Gaussian distribution, the model estimates $\mathcal{N}(\mu_{C_u}, \Sigma_{C_u})$ from the selected sparse codes. A sampled sparse code

$\tilde{n}_j$ is used to generate a new image:

$$X_j^{C_u} = G \left(\hat{e}^{C_u} + \alpha A_{C_u} \tilde{n}_j, H(F_{k,i}, \alpha A_{C_u} \tilde{n}_j) \right). \quad (16)$$

Equation (16) defines the final FeatureSAGE synthesis step. The term $\hat{e}^{C_u}$ preserves category-relevant structure, $A_{C_u} \tilde{n}_j$ introduces category-irrelevant variation, α controls edit intensity, and $H(\cdot)$ provides the edited feature tensor used by the generator.

1.4.6 Feature Editing Decoder Block

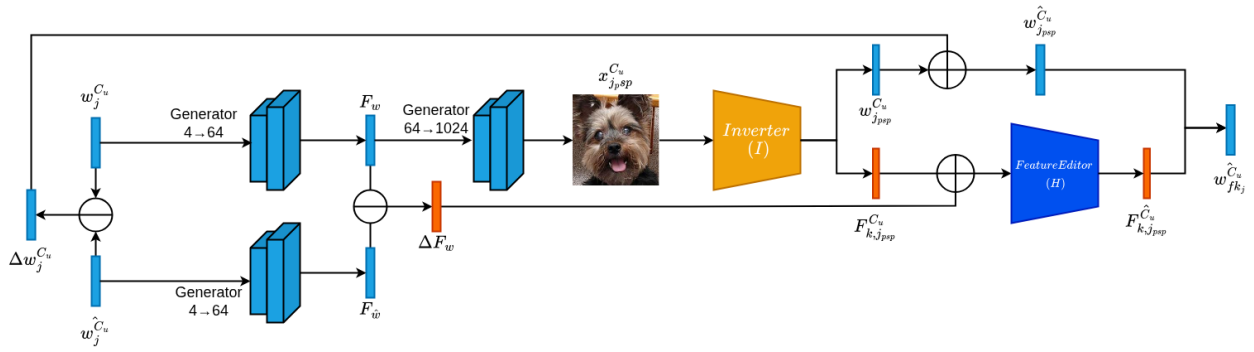


Figure 1-8. Feature Editing Decoder Block

Figure 1-8 shows the Feature Editing Decoder Block. The decoder receives the original latent representation, the edited latent representation from the adaptive editing block, and the intermediate feature tensor predicted by the StyleFeatureEditor Inverter. In this study, the feature tensor injected into the decoder is the layer-9 StyleGAN2 feature tensor, denoted as $F_{k,i}$ when $k = 9$.

The feature displacement used by the Feature Editor is computed as

$$\Delta F_w = F_w - F_{\hat{w}}. \quad (17)$$

Equation (17) compares the generator feature tensor of the original latent code with the generator feature tensor of the edited latent code. This displacement tells $H(\cdot)$ which feature-level details must be adjusted so that the image follows the latent edit while preserving

reconstruction detail.

The decoder therefore modifies the intermediate feature representation rather than replacing the full image. The edited feature tensor $H(F_{k,i}, \Delta F_w)$ is injected back into StyleGAN2 at layer 9, while the edited W^+ latent controls the global semantic direction of synthesis. This combination allows the generator to transform the input features into an output image with improved detail preservation and controlled attribute variation.

1.4.7 Training Strategy

In the training stage, StyleGAN2 is first trained with images from the seen categories. The Style-FeatureEditor Inverter and the pSp encoder are also trained. To reduce resource consumption, the proposed framework is trained in two phases.

1.4.8 Phase 1: Attribute Factorization Training

Phase 1 trains the attribute factorization block for disentangled sampling of category-irrelevant directions. The loss functions used in this phase are as follows:

- **Sparsity Loss (L_{sparse}):** This loss encourages each sparse code to activate only a few dictionary directions:

$$L_{\text{sparse}} = \|\sigma(\theta_0 n_i - \theta_1)\|_1. \quad (18)$$

In Equation (18), $\sigma(\cdot)$ is the sigmoid function, n_i is the sparse code, and θ_0 and θ_1 are hyperparameters that control sparsity. The sigmoid compresses the MLP output into a stable range before the L_1 penalty approximates the sparsity behavior of the L_0 objective in Equation (10).

- **Reconstruction Loss (L_{rec}):** This loss keeps the edited image close to the original input:

$$L_{\text{rec}} = \|G(e^{C_s} + An_i) - x_i^{C_s}\|_2. \quad (19)$$

In Equation (19), e^{C_s} is the class-center latent for the seen category, An_i is the attribute

offset, $G(\cdot)$ is the StyleGAN2 generator, and $x_i^{G_s}$ is the original image. The loss anchors editing so that category-irrelevant changes do not destroy the image content.

- **Orthogonality Loss (L_{orth}):**

This loss separates category-irrelevant directions from category-relevant directions:

$$L_{\text{orth}} = \|B^T A\|_F^2. \quad (20)$$

In Equation (20), B is the category-relevant embedding matrix, A is the category-irrelevant dictionary, and $\|\cdot\|_F^2$ is the squared Frobenius norm. The mathematical basis is orthogonality: directions in A should have minimal projection onto the category-relevant subspace B so that edits preserve class identity.

The total attribute factorization loss is

$$L_{\text{af}} = L_{\text{rec}} + \lambda_1 L_{\text{orth}} + \lambda_2 L_{\text{sparse}}, \quad (21)$$

where λ_1 and λ_2 weight the orthogonality and sparsity terms. Equation (21) balances reconstruction fidelity, disentanglement, and sparse semantic editing (?).

Optimization Algorithm

Training is done using the Adam optimizer with the same configuration as ? : a fixed learning rate of 1×10^{-4} , $\lambda_1 = 5 \times 10^{-4}$, $\lambda_2 = 5 \times 10^{-3}$, $\theta_0 = 5 \times 10^{-1}$, and $\theta_1 = -1$.

1.4.9 Phase 2: Feature Editor Training

Phase 2 trains the Feature Editor H using editing pairs generated with a pretrained pSp encoder. Following ?, the model generates an image pair $(X, \hat{X})$: pSp takes input image X and predicts w , which is edited by a sampled direction using the adaptive editing method from SAGE. This process allows the Feature Editor to learn feature changes associated with category-irrelevant attributes. To avoid degrading output quality when training only on

synthetic edits, H is also trained with a classical inversion task where the edit displacement is set to zero.

The editing loss is defined as

$$L_{\text{edit}} = L_2 + \lambda_{\text{lips}} L_{\text{lips}}. \quad (22)$$

The inversion loss uses the same reconstruction and perceptual terms:

$$L_{\text{inv}} = L_2 + \lambda_{\text{lips}} L_{\text{lips}}. \quad (23)$$

Equations (22) and (23) combine pixel-level reconstruction through L_2 with perceptual similarity through LPIPS. The total Feature Editor loss is

$$L_{\text{fe}} = L_{\text{edit}}(X, \hat{X}) + L_{\text{inv}}(X, \hat{X}). \quad (24)$$

Equation (24) jointly trains the Feature Editor to handle both edited outputs and identity-preserving inversion outputs.

The model is trained using the Adam optimizer with a fixed learning rate of 1×10^{-4} for 15,000 iterations on a single V100 16 GB vGPU. Mixed precision is enabled to improve memory efficiency due to multiple generator passes. The encoder is optimized using a weighted combination of reconstruction and perceptual losses with $\lambda_{l_2} = 10$ and $\lambda_{\text{lips}} = 0.8$, with an additional LPIPS scaling factor of 0.5. The model is initialized with a fixed length of 100 for the sparse dictionary A .

1.5 Evaluation Protocol

1.5.1 Evaluation Metrics

The models are evaluated using quantitative metrics for reconstruction quality, distributional alignment, and perceptual variation under the one-shot test-time setup. The same metric

implementations and evaluation parameters are used for AGE, SAGE, and FeatureSAGE.

1. Fréchet Inception Distance (FID)

Fréchet Inception Distance (FID) is used to evaluate the distributional similarity between generated images and real images (??). Instead of comparing images directly at the pixel level, FID compares high-level features extracted from a pretrained Inception-v3 network. This makes the metric suitable for measuring both image fidelity and sample diversity in few-shot image generation (?). The real and generated feature distributions are modeled as multivariate Gaussian distributions, and FID is computed as

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}). \quad (25)$$

In Equation (25), μ_r and Σ_r are the mean and covariance of the Inception features of real images, while μ_g and Σ_g are the mean and covariance of the Inception features of generated images. The term $\|\mu_r - \mu_g\|_2^2$ measures the squared Euclidean distance between feature means, $\text{Tr}(\cdot)$ is the trace operator, and $(\Sigma_r \Sigma_g)^{1/2}$ is the matrix square root of the covariance product.

Equation (25) computes the Fréchet, or 2-Wasserstein, distance between the two Gaussian feature distributions (?). A lower FID indicates that generated images are closer to the real image distribution in terms of quality and diversity. An FID of 0 indicates perfect distributional agreement.

2. Learned Perceptual Image Patch Similarity (LPIPS)

Learned Perceptual Image Patch Similarity (LPIPS) evaluates perceptual similarity by comparing deep feature activations from a pretrained neural network, such as VGG or AlexNet, rather than comparing raw pixels (?). The LPIPS score between images x and x' is computed as

$$\text{LPIPS}(x, x') = \sum_l \frac{1}{H_l W_l} \sum_{h=1}^{H_l} \sum_{w=1}^{W_l} \|w_l \odot (\hat{y}_{hw}^l(x) - \hat{y}_{hw}^l(x'))\|_2^2. \quad (26)$$

In Equation (26), $\hat{y}_{hw}^l(x)$ is the unit-normalized feature vector of image x at layer l and spatial position (h, w) , w_l is the learned channel-weight vector for layer l , and H_l and W_l are the height and width of the feature map at layer l .

LPIPS can be interpreted according to its evaluation context. For paired reconstruction, lower LPIPS indicates stronger perceptual similarity to the reference image. For generated samples in few-shot image generation, LPIPS may also be used to describe perceptual diversity among outputs; in that context, higher LPIPS indicates greater variation between generated images. This study reports LPIPS alongside FID so that improvements in distributional quality can be evaluated together with perceptual variation.

1.5.2 Baselines for Comparison

To contextualize the performance of the proposed few-shot image generation model, the study compares FeatureSAGE against two editing-based baseline methods:

- **AGE (?)** – Leverages latent-space attribute editing on pretrained GANs to produce novel-class images without retraining by disentangling class-relevant and category-irrelevant features.
- **SAGE (?)** – Builds on AGE by aggregating support images to create stable latent embeddings and by injecting class-consistent diversity through attribute mixing.

1.6 Expected Outcome

Based on the research design, FeatureSAGE is expected to improve latent code retrieval from real-world images by replacing the pSp-based inversion path of SAGE with the StyleFeatureEditor Inverter and Feature Editor. The expected technical effect is stronger reconstruction

and more stable editing because the framework represents each image using both W^+ latent codes and intermediate feature tensors. The W^+ code supports semantic attribute manipulation, while the feature tensor preserves fine visual details that may be weakened by latent-only inversion.

The framework is evaluated on the FUNIT Animal Faces and 102 Flowers datasets under the one-shot setting. Performance is assessed using Fréchet Inception Distance (FID) for distributional alignment and Learned Perceptual Image Patch Similarity (LPIPS) for perceptual variation. These metrics are used consistently across AGE, SAGE, and FeatureSAGE to determine whether the proposed inversion and feature-editing changes improve generated image quality without removing useful output diversity.

The expected measurable outcomes are as follows:

1. Lower FID, reflecting improved visual realism and closer distributional alignment between generated and real images.
2. Comparable LPIPS, reflecting that improvements in FID should not come from collapsing outputs into overly similar samples.

Together, these outcomes support the study’s goal of evaluating whether feature-level inversion and editing can strengthen SAGE for few-shot image generation and generative data augmentation in resource-limited settings.

CHAPTER 2. RESULTS AND DISCUSSION

This chapter presents the experimental results of the proposed FeatureSAGE framework and compares its performance with AGE and SAGE on one-shot image generation tasks. The experiments are conducted on two datasets, FUNIT Animal Faces and 102 Flowers, using a single reference image per target class. The reported evaluation is performed at 1024×1024 resolution. Image quality and distributional alignment are measured using Fréchet Inception Distance (FID), while Learned Perceptual Image Patch Similarity (LPIPS) is reported to describe perceptual variation among generated outputs. The comparison evaluates whether replacing the pSp-based SAGE inversion path with the StyleFeatureEditor Inverter and Feature Editor improves few-shot generation quality while maintaining usable edit diversity.

2.1 Quantitative Results

2.1.1 Quantitative Comparison Results between FeatureSAGE and Baselines

Resolution	Model	Flowers		Animal Faces	
		FID ↓	LPIPS	FID ↓	LPIPS
1024	AGE (Baseline)	61.57	0.4108	62.13	0.5033
1024	SAGE (Baseline)	54.17	0.4528	60.92	0.4867
1024	FeatureSAGE (Ours)	45.60	0.4209	49.60	0.4811

Table 2-1. Quantitative comparison of AGE, SAGE, and the proposed FeatureSAGE across Flowers and Animal Faces datasets at 1024×1024 resolution. Lower FID indicates better distributional alignment; LPIPS is reported as a perceptual variation measure.

2.2 Analysis of Quantitative Results

This section analyzes the quantitative performance trends observed in Table 2-1, focusing on how the proposed FeatureSAGE model compares against AGE and SAGE baselines across the

Flowers and Animal Faces datasets at 1024×1024 resolution using FID and LPIPS metrics.

2.2.1 102 Flowers

On a high-resolution setting, 1024×1024 , FeatureSAGE obtains an FID of 45.60, which corresponds to a 15.8% improvement over SAGE (54.17) and a 25.9% improvement over AGE (61.57). This substantial FID reduction demonstrates improved distributional alignment with the real flower images in the Inception feature space. In terms of perceptual diversity, FeatureSAGE achieves an LPIPS of 0.4209, compared to SAGE (0.4528) and AGE (0.4108). This represents a 7.0% decrease relative to SAGE and a 2.5% increase relative to AGE. The LPIPS values across all three methods range from 0.4108 to 0.4528, indicating that perceptual diversity remains relatively consistent across the approaches. The results show that FeatureSAGE achieves FID improvements of 15.8–25.9% while maintaining LPIPS within 7.0% of the baseline methods.

2.2.2 Animal Faces

On the Animal Faces HQ dataset at 1024×1024 , FeatureSAGE achieved an FID of 49.60, outperforming SAGE (60.92) by 18.6% and AGE (62.13) by 20.2%. This reduction in FID demonstrates FeatureSAGE’s superior distributional alignment with the real data distribution in the Inception feature space. In terms of perceptual diversity, FeatureSAGE achieves an LPIPS of 0.4811, compared to SAGE (0.4867) and AGE (0.5033). This represents a 1.2% decrease relative to SAGE and a 4.4% decrease relative to AGE. The LPIPS differences across all three methods are relatively small, with values ranging from 0.4811 to 0.5033, indicating comparable levels of perceptual diversity in the VGG feature space. The results show that FeatureSAGE achieves substantial FID improvements, 18.6–20.2%, while maintaining LPIPS values within 4.4% of the baseline methods.

2.3 Qualitative Evaluation

2.3.1 Visual Comparison Across Models



Figure 2-1. Comparison across AGE, SAGE, and the proposed FeatureSAGE model under one-shot setting, where a single reference image is used to condition the model during inference.



Figure 2-2. Comparison across AGE, SAGE, and the proposed FeatureSAGE model under one-shot setting with a different image input.



Figure 2-3. Comparison across AGE, SAGE, and the proposed FeatureSAGE model on the Animal Faces dataset under one-shot setting. Each model generates two outputs per input image to demonstrate edit diversity and semantic consistency.



Figure 2-4. Comparison across AGE, SAGE, and the proposed FeatureSAGE model under one-shot setting with a different dog input image. Each model generates two outputs per input to illustrate diversity and semantic consistency.

2.3.2 Visual Observations

Flowers

Figures 2-1 and 2-2 describe the qualitative analysis of the results of the edited flowers based on the baseline models AGE and SAGE, and the proposed model FeatureSAGE. The models are compared with regard to their ability to maintain the colors properly, the sharpness of the images, the level of contrast, and realism. As shown in Figure 2-1, the proposed model performs well in the retention of the pink petal colors from the input images. It also performs much better based on the level of semantic consistency. Note that the proposed model also fails to represent the combined red and yellow colors of the pistil from the input images. On the other hand, the proposed model performs much better than both the input images in the sharpness of the petal. Although there are inconsistencies from the input images, the proposed model also performs much better. In addition, the level of contrast in the proposed model results in the sharpest flower among all the models.

Further, the advantages of FeatureSAGE are highlighted by Figure 2-2. The sharpness in petal edges is better retained by the model compared to AGE and SAGE, which allows for a more photo-realistic and organic rendering. More varied outputs have also resulted from FeatureSAGE; for instance, in one generated image, the flower is presented with a slight tilt, reflecting strong attribute manipulation. The model maintains higher clarity, with balanced contrast and natural color distribution, while AGE slightly looks pale with reduced contrast in areas, which is somewhat settled by SAGE as it balances the contrast. Overall,

the generated flower images produced by FeatureSAGE are the sharpest, most realistic, and visually consistent as compared to the other models being tested.

Animal Faces

Figure 2-3 provides a comparison of the outputs produced by the AGE, SAGE, and proposed FeatureSAGE model on the Animal Faces data set using a picture of a dog as input. As observed, the outputs produced by the models have managed to maintain the color and texture of the input data, implying success by these models in maintaining basic characteristics produced by each model. Considering the clarity and image properties, all the outputs produced by these models can be considered similar, since there is minimal blurring effect and texture preservation on the fur of the input image. Consistency and overall properties, on the other hand, differ significantly, since there is more diversity apparent in the outputs produced by the AGE and SAGE models, with a slight difference in orientation between the two outcomes produced by each model. The more consistent results produced by the proposed FeatureSAGE model, on the contrary, demonstrate success toward maintaining integrity while providing semantic edits. Also, there are some minor enhancements in the realism of the output, whereby the output from FeatureSAGE shows slightly more coherent shading and blending of the fur patterns. This indicates that although all these models demonstrate the ability to sustain basic attributes like color and texture, the FeatureSAGE model has the best consistency in output.

Figure 2-4 shows similar behavior to the third image input for FeatureSAGE, where it maintains consistency in terms of pose within the results, showing that it does not have great variability compared to AGE and SAGE, which have more diverse orientations. At the same time, it may be noted that FeatureSAGE is better at maintaining consistency of shape and structure for eyes, thus improving realism. It is also observed that the results from FeatureSAGE have better clarity and contrast, improving image quality, particularly for fur texture and shading. On the other hand, it may be noted that AGE performs best in terms of

diversity among all models, particularly for pose variation. Yet, it is observed that this diversity may also decrease image clarity.

2.4 Failure Cases and Limitations

During experimentation and evaluation of the FeatureSAGE framework, several limitations were observed:

1. **Dependence on pretrained StyleGAN2.** FeatureSAGE uses a pretrained StyleGAN2 generator and therefore remains limited to domains where a suitable generator is available. If the base generator does not adequately represent certain visual categories or variations, those limitations can carry over to the generated results. This dependence also means that biases or weaknesses in the pretrained latent space may affect attribute editing behavior.
2. **Computational overhead.** Adding the StyleFeatureEditor Inverter and Feature Editor increases computation relative to the base SAGE setup. The additional inversion, feature extraction, and feature-injection steps improve reconstruction detail, but they also require more GPU memory and increase processing time per operation. This trade-off is important when scaling to larger datasets or time-sensitive applications.

These limitations indicate that FeatureSAGE improves image quality and stability at the cost of a heavier processing pipeline. Future work should therefore examine computational optimization, alternative base generators, and multi-level feature editing strategies while preserving the same goal of stable few-shot attribute group editing.

2.5 Chapter Summary

The results show that FeatureSAGE achieved the lowest FID on both evaluated datasets. On 102 Flowers, FeatureSAGE obtained an FID of 45.60, improving over SAGE by 15.8% and over

AGE by 25.9%. On Animal Faces, FeatureSAGE obtained an FID of 49.60, improving over SAGE by 18.6% and over AGE by 20.2%. These outcomes indicate stronger distributional alignment with the real image data after integrating the StyleFeatureEditor Inverter and Feature Editor into the SAGE pipeline. LPIPS remained within a narrow range relative to the baselines, with 0.4209 on Flowers and 0.4811 on Animal Faces, indicating that the FID gains did not remove perceptual variation from the generated outputs. Qualitative comparisons further show that FeatureSAGE produced sharper flower structures, more coherent color and contrast, and more stable animal-face structure, although the method still carries computational overhead and remains dependent on the pretrained StyleGAN2 generator.

CHAPTER 3. SUMMARY, CONCLUSION AND RECOMMENDATION

3.1 Summary of Findings

This study evaluated whether integrating the StyleFeatureEditor Inverter and Feature Editor into SAGE improves few-shot image generation under a one-shot setting. The findings are summarized as follows:

3.1.1 Integration of the StyleFeatureEditor Inverter

FeatureSAGE replaces the pSp-based inversion path of SAGE with the StyleFeatureEditor Inverter, allowing the framework to represent each image through both W^+ latent codes and intermediate feature tensors. This directly addresses the research problem concerning inversion fidelity and detail preservation. At 1024×1024 resolution, FeatureSAGE achieved an FID of 45.60 on 102 Flowers and 49.60 on Animal Faces.

3.1.2 Use of the Feature Editor for feature-level detail preservation

The Feature Editor modifies intermediate feature tensors together with the edited latent code. This supports the decoder block by preserving local detail while allowing semantic attribute edits. Qualitative comparisons show sharper petal structures, stronger contrast, and more coherent animal-face structure for FeatureSAGE compared with AGE and SAGE.

3.1.3 Quantitative improvement over AGE and SAGE

FeatureSAGE obtained the best FID score on both evaluated datasets. On 102 Flowers, it improved FID by 15.8% over SAGE and 25.9% over AGE. On Animal Faces, it improved FID by 18.6% over SAGE and 20.2% over AGE. These results indicate stronger distributional alignment between generated and real images.

3.1.4 Preservation of perceptual variation

The reported LPIPS values remained close to the baselines. FeatureSAGE obtained an LPIPS of 0.4209 on 102 Flowers, compared with 0.4528 for SAGE and 0.4108 for AGE. On Animal Faces, FeatureSAGE obtained an LPIPS of 0.4811, compared with 0.4867 for SAGE and 0.5033 for AGE. These values indicate that the FID improvements were achieved while maintaining comparable perceptual variation across generated outputs.

3.2 Limitations

Though FeatureSAGE holds promise in attribute group editing and limited example image synthesis, some constraints observed during testing indicate that there is scope for further research. The first is the reliance on a pre-trained StyleGAN2 generator, which restricts the application of this tool only to domains where such generators have been developed, and any biases in the model may have been passed on to the output. The second is the potential performance impact of integrating the StyleFeatureEditor with the Feature Editor, which increases computational requirements. Future research could focus on other base models, incorporate a multi-level feature approach to improve consistency between edits, optimize computation for more general or faster models, and extend few-shot adaptation to work well even on entirely new classes.

3.3 Recommendations

Future research should focus on reducing the computational overhead introduced by the StyleFeatureEditor Inverter and Feature Editor. Since FeatureSAGE requires additional feature extraction, feature editing, and feature injection steps, optimization should target faster inference and lower GPU memory usage while preserving the reconstruction and editing advantages observed in this study.

Further work may also examine alternative base generators beyond StyleGAN2. Because

FeatureSAGE depends on the availability and quality of a pretrained generator, testing other generator backbones could clarify how much of the framework’s performance depends on the StyleGAN2 latent and feature spaces. Another direction is to study multi-level feature editing, where feature tensors from more than one generator layer are considered, to determine whether consistency between fine details and higher-level structure can be improved without increasing instability.

3.4 Conclusion

This study addressed the problem of improving few-shot image generation in SAGE by replacing its pSp-based inversion path with a StyleFeatureEditor Inverter and Feature Editor. The original research problem asked whether improved inversion and feature-level editing could preserve image details, support stable semantic edits, and improve generated image quality for novel classes in the Flowers and Animal Faces datasets. Based on the reported results, the answer is affirmative: FeatureSAGE achieved lower FID than both AGE and SAGE on the two evaluated datasets while maintaining comparable LPIPS values.

The first objective, integrating StyleFeatureEditor into SAGE, was achieved by incorporating the StyleFeatureEditor Inverter and Feature Editor into the FeatureSAGE architecture. The second objective, evaluating reconstruction and generation quality using FID and LPIPS, was addressed through the quantitative evaluation at 1024×1024 resolution. FeatureSAGE achieved an FID of 45.60 and LPIPS of 0.4209 on 102 Flowers, and an FID of 49.60 and LPIPS of 0.4811 on Animal Faces. The third objective, benchmarking against AGE and SAGE, was also achieved: FeatureSAGE improved FID by 15.8% over SAGE and 25.9% over AGE on 102 Flowers, and by 18.6% over SAGE and 20.2% over AGE on Animal Faces.

Overall, the study contributes an enhanced SAGE-based framework that combines latent-space attribute editing with feature-level detail preservation. The findings show that this integration improves distributional alignment and qualitative image coherence in one-shot generation while retaining perceptual variation. However, the framework remains dependent

on a pretrained StyleGAN2 generator and requires greater computation because of the additional inversion and feature-editing stages.